Best Seat

AlgoExpert · Medium · Arrays

The problem

- A row of seats (1 taken, 0 free; both ends taken).
- Return the index of the free seat with the most space on both sides (centre of the widest gap, leftmost on ties), or -1.

Intuition

- The best seat is the middle of the longest run of empty seats between occupied ones.

Approach

- 1. Scan for runs of zeros between ones
- 2. Track the longest run
- 3. Return its middle index, or minus one if none

Java solution

```java
public class BestSeat {
    public static int bestSeat(int[] seats) {
        int best = -1, bestGap = 0, l = 0;
        while (l < seats.length) {
            int r = l + 1;
            while (r < seats.length && seats[r] == 0) r++;
            int gap = r - l - 1;
            if (gap > bestGap) { bestGap = gap; best = (l + r) / 2; }
            l = r;
        }
        return best;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Recap

- The best seat is the middle of the longest run of empty seats between occupied ones.
- Complexity: O(n)